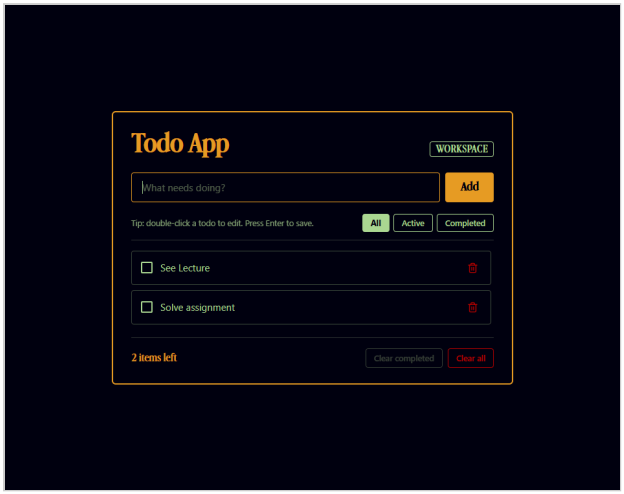
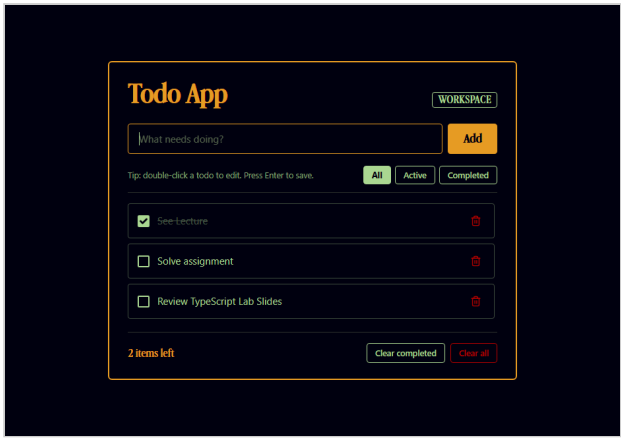
Web Development CSE343: Lab #4

Amir Atef Abdullah - 24P0310

Screenshots





Source Code Listings

page.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Todo App</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="todo-container">
        <header class="todo-header">
            <h1 class="todo-title">Todo App</h1>
            <span class="todo-tag">Workspace</span>
        </header>

        <form id="todo-form" class="todo-form">
            <input type="text" id="todo-input" class="todo-input" placeholder="What needs doing?" autof
            <button type="submit" id="add-btn" class="btn-add">Add</button>
        </form>

        <div class="todo-toolbar">
            <span class="todo-tip">Tip: double-click a todo to edit. Press Enter to save.</span>
            <div class="filter-group">
                <button type="button" class="filter-btn active" data-filter="all">All</button>
                <button type="button" class="filter-btn" data-filter="active">Active</button>
                <button type="button" class="filter-btn" data-filter="completed">Completed</button>
            </div>
        </div>

        <ul id="todo-list" class="todo-list"></ul>

        <footer class="todo-footer">
            <span id="items-left" class="items-left">0 items left</span>
            <div class="footer-actions">
                <button type="button" id="clear-completed-btn" class="btn-footer">Clear completed</butt
                <button type="button" id="clear-all-btn" class="btn-footer">Clear all</button>
            </div>
        </footer>
    </div>

    <script src="app.js"></script>
</body>
</html>
```

Source Code Listings

`style.css`

```
@font-face {
    font-family: 'Advercase';
    src: url('Font(s)/indieground-advercasefont-demo/AdvercaseFont-Demo-Regular.otf') format('opentype'
    font-weight: 400;
}

@font-face {
    font-family: 'Advercase';
    src: url('Font(s)/indieground-advercasefont-demo/AdvercaseFont-Demo-Bold.otf') format('opentype');
    font-weight: 700;
}

:root {
    --guardsman-red: #BA0101;
    --golden-bell: #E99D25;
    --pale-olive-green: #ADD794;
    --jaguar: #040011;
    --font-heading: 'Advercase', Georgia, serif;
    --font-body: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Arial, sans-serif;
}

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    background-color: var(--jaguar);
    font-family: var(--font-body);
    color: var(--pale-olive-green);
    padding: 24px 16px;
}

.todo-container {
    width: 100%;
    max-width: 620px;
    background-color: var(--jaguar);
    border: 2px solid var(--golden-bell);
    border-radius: 6px;
    padding: 32px 28px 24px;
}

.todo-header {
    display: flex;
    align-items: baseline;
    justify-content: space-between;
    margin-bottom: 24px;
}

.todo-title {
    font-family: var(--font-heading);
    font-size: 34px;
    color: var(--golden-bell);
    letter-spacing: 0.5px;
}

.todo-tag {
    font-family: var(--font-heading);
    font-size: 13px;
    letter-spacing: 1px;
    color: var(--pale-olive-green);
    border: 1px solid var(--pale-olive-green);
    padding: 3px 8px;
    border-radius: 4px;
}

.todo-form {
```

```
    display: flex;
    gap: 8px;
    margin-bottom: 18px;
}

.todo-input {
    flex: 1;
    background-color: var(--jaguar);
    border: 1px solid var(--pale-olive-green);
    border-radius: 4px;
    padding: 12px 16px;
    font-size: 15px;
    font-family: var(--font-body);
    color: var(--pale-olive-green);
    outline: none;
}

.todo-input::placeholder {
    color: var(--pale-olive-green);
    opacity: 0.45;
}

.todo-input:focus {
    border-color: var(--golden-bell);
}

.btn-add {
    background-color: var(--golden-bell);
    color: var(--jaguar);
    font-family: var(--font-heading);
    font-size: 15px;
    font-weight: 700;
    padding: 0 22px;
    border: 1px solid var(--golden-bell);
    border-radius: 4px;
    cursor: pointer;
}

.btn-add:hover {
    background-color: var(--guardsman-red);
    border-color: var(--guardsman-red);
    color: var(--golden-bell);
}

.todo-toolbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 12px;
    margin-bottom: 16px;
    padding-bottom: 12px;
    border-bottom: 1px solid rgba(173, 215, 148, 0.25);
}

.todo-tip {
    font-size: 12.5px;
    opacity: 0.75;
}

.filter-group {
    display: flex;
    gap: 6px;
}

.filter-btn {
    background-color: transparent;
    color: var(--pale-olive-green);
    border: 1px solid var(--pale-olive-green);
    border-radius: 4px;
    padding: 5px 12px;
    font-size: 12.5px;
    cursor: pointer;
}

.filter-btn:hover {
    border-color: var(--golden-bell);
    color: var(--golden-bell);
}

.filter-btn.active {
    background-color: var(--pale-olive-green);
    color: var(--jaguar);
```

```
    font-weight: 700;
}

.todo-list {
    list-style: none;
    display: flex;
    flex-direction: column;
    gap: 8px;
    margin-bottom: 20px;
    max-height: 420px;
    overflow-y: auto;
}

.todo-item {
    background-color: var(--jaguar);
    border: 1px solid rgba(173, 215, 148, 0.35);
    border-radius: 4px;
}

.todo-item:hover {
    border-color: var(--golden-bell);
}

.todo-view {
    display: flex;
    align-items: center;
    padding: 12px 14px;
    gap: 12px;
}

.todo-item.editing .todo-view {
    display: none;
}

.todo-item.editing .todo-edit-input {
    display: block;
}

.todo-checkbox-container {
    display: inline-flex;
    align-items: center;
    cursor: pointer;
    user-select: none;
}

.todo-checkbox {
    position: absolute;
    opacity: 0;
}

.custom-checkbox {
    width: 18px;
    height: 18px;
    border-radius: 2px;
    border: 2px solid var(--pale-olive-green);
    background-color: var(--jaguar);
    display: flex;
    align-items: center;
    justify-content: center;
}

.custom-checkbox svg {
    display: none;
    width: 12px;
    height: 12px;
    stroke: var(--jaguar);
    stroke-width: 2.8;
    fill: none;
}

.todo-checkbox:checked + .custom-checkbox {
    background-color: var(--pale-olive-green);
}

.todo-checkbox:checked + .custom-checkbox svg {
    display: block;
}

.todo-text {
    flex: 1;
    font-size: 15px;
    word-break: break-word;
```

```
    cursor: pointer;
}

.todo-item.completed .todo-text {
    opacity: 0.38;
    text-decoration: line-through;
}

.todo-edit-input {
    display: none;
    width: 100%;
    padding: 12px 14px;
    font-size: 15px;
    font-family: var(--font-body);
    background-color: var(--jaguar);
    border: 2px solid var(--golden-bell);
    border-radius: 4px;
    color: var(--pale-olive-green);
    outline: none;
}

.btn-delete {
    background-color: transparent;
    border: none;
    cursor: pointer;
    color: var(--guardsman-red);
    display: flex;
    align-items: center;
    padding: 4px;
}

.btn-delete svg {
    width: 17px;
    height: 17px;
    stroke: currentColor;
    fill: none;
    stroke-width: 2;
}

.btn-delete:hover {
    background-color: var(--guardsman-red);
    color: var(--jaguar);
    border-radius: 2px;
}

.todo-empty {
    text-align: center;
    padding: 24px 16px;
    opacity: 0.6;
    font-size: 14px;
    border: 1px dashed rgba(173, 215, 148, 0.35);
    border-radius: 4px;
}

.todo-footer {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 12px;
    padding-top: 16px;
    border-top: 1px solid rgba(173, 215, 148, 0.25);
}

.items-left {
    font-size: 13.5px;
    color: var(--golden-bell);
    font-family: var(--font-heading);
}

.footer-actions {
    display: flex;
    gap: 8px;
}

.btn-footer {
    background-color: transparent;
    border-radius: 4px;
    padding: 6px 12px;
    font-size: 12.5px;
    cursor: pointer;
}

```

```
#clear-completed-btn {
    border: 1px solid var(--pale-olive-green);
    color: var(--pale-olive-green);
}

#clear-completed-btn:hover:not(:disabled) {
    background-color: var(--pale-olive-green);
    color: var(--jaguar);
}

#clear-all-btn {
    border: 1px solid var(--guardsman-red);
    color: var(--guardsman-red);
}

#clear-all-btn:hover:not(:disabled) {
    background-color: var(--guardsman-red);
    color: var(--golden-bell);
}

.btn-footer:disabled {
    opacity: 0.3;
    cursor: not-allowed;
}

@media (max-width: 540px) {
    .todo-container {
        padding: 20px 16px 18px;
    }
    .todo-toolbar, .todo-footer {
        flex-direction: column;
        align-items: flex-start;
        gap: 10px;
    }
    .footer-actions {
        width: 100%;
        justify-content: flex-end;
    }
}
```

Source Code Listings

app.ts

```
interface Todo {
    id: string;
    text: string;
    completed: boolean;
}

class TodoApp {
    private todos: Todo[] = [];
    private filter: 'all' | 'active' | 'completed' = 'all';
    private editingId: string | null = null;
    private readonly storageKey = 'lab4_todos';

    private input = document.getElementById('todo-input') as HTMLInputElement;
    private list = document.getElementById('todo-list') as HTMLUListElement;
    private itemsLeft = document.getElementById('items-left') as HTMLElement;
    private clearCompletedBtn = document.getElementById('clear-completed-btn') as HTMLButtonElement;
    private clearAllBtn = document.getElementById('clear-all-btn') as HTMLButtonElement;

    constructor() {
        this.load();
        this.bindEvents();
        this.render();
    }

    private load(): void {
        const raw = localStorage.getItem(this.storageKey);
        this.todos = raw ? JSON.parse(raw) : [
            { id: '1', text: 'See Lecture', completed: false },
            { id: '2', text: 'Solve assignment', completed: false }
        ];
    }

    private save(): void {
        localStorage.setItem(this.storageKey, JSON.stringify(this.todos));
    }

    private get filtered(): Todo[] {
        if (this.filter === 'active') return this.todos.filter(t => !t.completed);
        if (this.filter === 'completed') return this.todos.filter(t => t.completed);
        return [...this.todos];
    }

    private render(): void {
        const activeCount = this.todos.filter(t => !t.completed).length;
        const completedCount = this.todos.length - activeCount;

        this.itemsLeft.textContent = `${activeCount} item${activeCount === 1 ? '' : 's'} left`;
        this.clearCompletedBtn.disabled = completedCount === 0;
        this.clearAllBtn.disabled = this.todos.length === 0;

        if (this.filtered.length === 0) {
            this.list.innerHTML = '<li class="todo-empty">No tasks</li>';
            return;
        }

        this.list.innerHTML = this.filtered.map(({ id, text, completed }) => `
            <li class="todo-item${completed ? ' completed' : ''}${this.editingId === id ? ' editing' :
                <div class="todo-view">
                    <label class="todo-checkbox-container">
                        <input type="checkbox" class="todo-checkbox" ${completed ? 'checked' : ''}>
                        <span class="custom-checkbox">
                            <svg viewBox="0 0 12 10"><polyline points="1.5 5 4.5 8 10.5 2"></polyline><
                        </span>
                    </label>
                    <span class="todo-text">${text}</span>
                    <button type="button" class="btn-delete" title="Delete todo">
                        <svg viewBox="0 0 24 24"><polyline points="3 6 5 6 21 6"></polyline><path d="M1
                    </button>
                </div>
                <input type="text" class="todo-edit-input" value="${text}">
            </li>
        `).join('');
```

```
73
74          if (this.editingId) {
75              const editInput = this.list.querySelector<HTMLInputElement>(`.todo-item[data-id="${this.edi
76              if (editInput) {
77                  editInput.focus();
78                  editInput.select();
79              }
80          }
81      }
82
83      private bindEvents(): void {
84          document.getElementById('todo-form')?.addEventListener('submit', (e) => {
85              e.preventDefault();
86              const val = this.input.value.trim();
87              if (val) {
88                  this.todos.push({ id: Date.now().toString(), text: val, completed: false });
89                  this.input.value = '';
90                  this.save();
91                  this.render();
92              }
93          });
94
95          document.querySelectorAll('.filter-btn').forEach(btn => {
96              btn.addEventListener('click', () => {
97                  this.filter = (btn.getAttribute('data-filter') as 'all' | 'active' | 'completed') || 'a
98                  document.querySelectorAll('.filter-btn').forEach(b => b.classList.toggle('active', b ==
99                  this.render();
100             });
101         });
102
103         this.clearCompletedBtn.addEventListener('click', () => {
104             this.todos = this.todos.filter(t => !t.completed);
105             this.save();
106             this.render();
107         });
108
109         this.clearAllBtn.addEventListener('click', () => {
110             this.todos = [];
111             this.save();
112             this.render();
113         });
114
115         this.list.addEventListener('change', (e) => {
116             const target = e.target as HTMLInputElement;
117             const item = target.closest('.todo-item') as HTMLElement | null;
118             if (item?.dataset.id) {
119                 const todo = this.todos.find(t => t.id === item.dataset.id);
120                 if (todo) {
121                     todo.completed = target.checked;
122                     this.save();
123                     this.render();
124                 }
125             }
126         });
127
128         this.list.addEventListener('click', (e) => {
129             const target = e.target as HTMLElement;
130             const delBtn = target.closest('.btn-delete');
131             const item = target.closest('.todo-item') as HTMLElement | null;
132             if (delBtn && item?.dataset.id) {
133                 this.todos = this.todos.filter(t => t.id !== item.dataset.id);
134                 this.save();
135                 this.render();
136             }
137         });
138
139         this.list.addEventListener('dblclick', (e) => {
140             const target = e.target as HTMLElement;
141             if (target.classList.contains('todo-text')) {
142                 const item = target.closest('.todo-item') as HTMLElement | null;
143                 if (item?.dataset.id) {
144                     this.editingId = item.dataset.id;
145                     this.render();
146                 }
147             }
148         });
149
150         const commitEdit = (input: HTMLInputElement) => {
151             const item = input.closest('.todo-item') as HTMLElement | null;
152             if (item?.dataset.id && this.editingId === item.dataset.id) {
153                 const val = input.value.trim();
154                 if (val) {
```

```
                    const todo = this.todos.find(t => t.id === item.dataset.id);
                    if (todo) todo.text = val;
                } else {
                    this.todos = this.todos.filter(t => t.id !== item.dataset.id);
                }
                this.editingId = null;
                this.save();
                this.render();
            }
        };

        this.list.addEventListener('keydown', (e) => {
            const input = e.target as HTMLInputElement;
            if (input.classList.contains('todo-edit-input')) {
                if (e.key === 'Enter') {
                    commitEdit(input);
                } else if (e.key === 'Escape') {
                    this.editingId = null;
                    this.render();
                }
            }
        });

        this.list.addEventListener('focusout', (e) => {
            const input = e.target as HTMLInputElement;
            if (input.classList.contains('todo-edit-input')) {
                commitEdit(input);
            }
        });
    }
}

new TodoApp();

```

Source Code Listings

`tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ES2020",
    "strict": true,
    "noEmitOnError": true
  },
  "include": ["*.ts"]
}
```

repo link:
https://github.com/Auipp/Web-Dev-Labs-Repo